AeroManten — Documentación del sistema

Estructura técnica: lenguaje, hosting, base de datos y modelo de datos

Lenguaje y framework

- TypeScript de punta a punta (frontend y backend en el mismo proyecto).
- Next.js 16.3 (App Router), con React 19. El build de producción usa webpack explícitamente (next build --webpack) porque Turbopack tenía problemas en el hosting anterior; en desarrollo local corre con Turbopack.
- Server Actions de Next.js para todas las operaciones (crear/editar/borrar) — no hay una API REST separada, los formularios llaman directo a funciones del servidor ("use server").
- Tailwind CSS v4 para los estilos, con una paleta de colores propia (celeste, gris, marrón) definida en globals.css.

Base de datos

- MySQL 8, accedida vía Prisma ORM 6.19 (no se usan migraciones formales, sino prisma db push: el schema define la verdad y se sincroniza directo contra la base).
- Base local (aeromanten_local): corre en XAMPP en la PC de desarrollo.
- Base de producción/prueba (aeromanten_test en el droplet): donde vive la carga real de datos, servida en doliarg.com.ar.

Autenticación y permisos

- NextAuth v5 (Auth.js), login por usuario/contraseña (bcrypt), sesión JWT.
- Rol (USER / ADMIN): ADMIN tiene acceso total siempre; solo ADMIN puede entrar a Personal, Grupos y Usuarios.
- Grupo (Insertar / Modificar / Borrar): para el resto de los usuarios, define qué pueden hacer en Propietarios, Aeronaves, Activos, Mantenimiento Preventivo, Medidores y Órdenes de Trabajo. Todos pueden ver y navegar; insertar, modificar o borrar depende de esos tres permisos.

Dónde vive el código y cómo se despliega

- Repositorio en GitHub (github.com/munditkd/AeroManten), rama main.
- Hosting: un droplet de DigitalOcean (Ubuntu 24.04, región NYC). Se abandonó el hosting compartido de Hostinger original por limitaciones de recursos que impedían hacer builds de forma confiable.
- En el droplet: Nginx como proxy reverso más certificado SSL (Let's Encrypt, con renovación automática), y PM2 manteniendo la app Node corriendo (reinicia sola si se cae o si reinicia el servidor).
- Flujo de deploy: los cambios se prueban localmente, se suben a GitHub, se bajan al droplet por SSH (git pull), se compilan (npm run build) y se reinicia la app con PM2.
- El dominio de prueba actual es doliarg.com.ar; aeromanten.com.ar (el dominio real) todavía no se migró ahí.

Los datos del sistema (entidades principales)

Comerciales / flota

- **Propietarios:** Dueños de aeronaves.
- **Aeronaves:** Matrícula, fabricante, N° de parte, horas/ciclos/meses TSN-TSO, y un código autonumérico (AN-001...) que por ahora no se usa para nada.

- **Activos:** Componentes montados en una aeronave o en depósito (motores, hélices, etc.), con su propio código autonumérico.

Mantenimiento

- **Mantenimiento Preventivo:** Planes de tareas periódicas (por horas, ciclos o meses).
- **Relación Activo/Aeronave con MP:** Vincula cada plan a un activo o una aeronave concreta, y guarda el historial de cada vez que se realizó.
- **Registros de mantenimiento:** Bitácora libre por aeronave o activo.
- **Órdenes de Trabajo:** El módulo más grande: código autonumérico (OT-00001...), estado, prioridad, fechas, originador/responsable, vinculación opcional a aeronave, activo o plan de MP, y una tabla de mano de obra con varias entradas por OT (operario, horas, tareas, costo).

Personal y accesos

- **Personal:** Legajo de la empresa (pilotos, mecánicos, operarios, administrativos), con habilitación y fecha de vencimiento.
- **Grupos:** Perfiles de permiso (Insertar / Modificar / Borrar).
- **Usuarios:** Las cuentas de login, cada una con su Rol, su Grupo asignado, y opcionalmente vinculada a una persona de Personal (para que las Órdenes de Trabajo sepan quién las creó).